Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский Политехнический университет Петра Великого
—
Институт компьютерных наук и кибербезопасности

КУРСОВАЯ РАБОТА

«Разработка синтаксического анализатора языка Bash»
по дисциплине «Формальные грамматики и теория компиляторов»

Выполнил Чупров Е.А.

студент гр. 5131001/30501

Руководитель

ст. преподаватель Семьянов П.В.

Санкт-Петербург
2026

ОГЛАВЛЕНИЕ

1 ЦЕЛЬ РАБОТЫ **3**
2 ХОД РАБОТЫ **4**
2.1 Теоретические сведения 4
2.2 Лексический анализатор языка 6
2.3 Синтаксический анализатор языка 7
2.4 Диагностические правила 18
2.5 Возникшие конфликты и их исправление 19
2.6 Работоспособность парсера 21
3 РЕЗУЛЬТАТЫ **25**
4 ВЫВОД **26**
ПРИЛОЖЕНИЕ 1 **27**
ПРИЛОЖЕНИЕ 2 **29**

1 ЦЕЛЬ РАБОТЫ

Получение навыков проектирования грамматик языков программирования, самостоятельная разработка контекстно-свободной грамматики подмножества языка Bash и создание его синтаксического распознавателя с использованием генераторов лексических и синтаксических анализаторов flex и yacc.

Распознаватель решает только вопрос принадлежности текста языку. Сценарий он не исполняет и подстановки не раскрывает.

2 ХОД РАБОТЫ

2.1 Теоретические сведения

Bash — командный интерпретатор GNU и язык сценариев UNIX. По синтаксису он продолжает Bourne shell и раздел Shell Command Language стандарта POSIX.

Весь Bash в одну контекстно-свободную грамматику не помещается. Знаки #, $, скобки и перевод строки в кавычках, в here-документе и в арифметике читаются по-разному. Алиасы, тильда и раскрытие {a,b} меняют текст ещё до разбора, поэтому в грамматику они не входят.

В работе взято подмножество языка. Файл начинается со строки #!/bin/bash. После пути можно оставить только пробелы и табы. Файл, состоящий только из шебанга, принимается. Пустой файл не принимается. Дальше идут обычные команды с аргументами и перенаправлениями >, >>, <, 2>, 2>&1 в конце. Присваивание NAME=value считается отдельной командой. Команды соединяются конвейером, &&, ||, ;, & и переводом строки. Составная команда — это if, while, until, for, select, case, группа { ...; } и подшелл (...).

Проверка пишется как [...]. Знаки вроде -lt остаются обычными словами. Есть if с elif и else, циклы while, until, for и select, и case. У for и select есть форма со списком in и форма без in. Каждая ветка case заканчивается ;;. Условие if, elif, while и until — одна команда, конвейер или цепочка && и ||, а не только проверка [...]. Запись if true; then языку принадлежит. Функция записывается как name() { ...; }, function name { ...; } или function name() { ...; }. Те же фигурные скобки могут быть отдельной командой. Перед } в непустой группе нужны ; или перевод строки. У local указываются имя, a= или a=1. Вне кавычек допускаются $имя, $1, $#, $@, $*, $$, $?, ${имя} и $(...). Внутри $(...) тот же список команд, но без шебанга. В кавычках $ не разбирается.

Не вошло то, чему нужен отдельный режим сканера или своя грамматика. Here-документ читает текст до слова-ограничителя, поэтому << отвергается с пояснением. Проверка [[...]] тоже отвергается с пояснением. Запись ((...)) без пробела — это вложенный подшелл, а не арифметика. $((...)) читается как подстановка команды. В ${...} можно написать лишь имя: без ${var:-default} и массивов. Обратные кавычки `cmd` не поддерживаются, вместо них используется $(...). Подстановка процесса <(...) не входит. Запись VAR=value command не входит. FOO=1 — присваивание, echo FOO=1 — команда с аргументом. Не принимаются шебанг через /usr/bin/env и лишний текст после пути. Перевод строки внутри одинарных кавычек не допускается. В двойных кавычках он допустим только после \. Вне кавычек \ не поддерживается, в том числе как перенос строки.

Часть отказов идёт от самих лексем. Слова done, in и select везде ключевые, поэтому echo done — ошибка. Вне кавычек нельзя поставить \. Скобки [ ] { } в слово не входят, так что ls [abc]* и echo {a,b} не принимаются. Одиночный ! остаётся обычным словом, а [не может быть аргументом, поэтому if ! [...] отвергается как синтаксическая ошибка.

2.2 Лексический анализатор языка

Лексический анализатор разбивает входной текст сценария на последовательность лексем. Сканер начинает в состоянии INITIAL и переходит в состояние BODY только после шебанга #!/bin/bash. Любой другой первый символ, лишний текст в строке шебанга и пустой файл дают ошибку на строке 1. Комментарии #.* обрабатываются только внутри состояния BODY.

Основные типы токенов и регулярные выражения приведены в таблице 1. Макросы flex записаны так же, как в lex_bash.l.

Категория	**Конструкции и токены**
Шебанг и комментарии	"#!/bin/bash"[\t]*\n, "#!/bin/bash"[\t]*, #.*
Ключевые слова	if, then, elif, else, fi, while, until, do, done, for, select, in, case, esac, function, local
Имена, слова, присваивание	ID — [A-Za-z_][A-Za-z0-9_]*. WORD — непробельная последовательность без операторов, кавычек и $ (точный класс записан в приложении 1). {ID}={WORD} и {ID}= возвращают ASSIGN
Переменные и подстановки	"$"{ID} (DOLLAR_ID), "$"[0-9]+ (DOLLAR_NUM), $# $@ $* $$ $? (DOLLAR_SPECIAL), "${"{ID}"}" (DOLLAR_BRACE), "$(" (DOLLAR_LPAREN)
Строковые литералы	' и " читаются функциями и возвращают STRING
Операторы и разделители	&&, ||, ;;, ;, &, |, <, >, >>, 2>, 2>&1, (,), {, }, [,], перевод строки (NL)

Таблица 1 – Ключевые конструкции лексического анализатора

Flex выбирает самое длинное совпадение, поэтому >> не распадается на два >, а 2>&1 — на 2> и отдельный фрагмент. Порядок правил решает только спор одинаковой длины, например ключевое слово и имя. Пробел и таб токенов не порождают. Перевод строки в теле программы — токен NL. При включённом %option yylineno flex увеличивает номер строки в момент чтения этого символа.

Полный код анализатора приведён в **Приложении 1**.

2.3 Синтаксический анализатор языка

Элемент	Описание
program	Вся программа: шебанг и список команд
list	Список команд с необязательными пустыми строками
stmt_seq	Последовательность конструкций and_or
separator	Разделитель команд: ;, & или перевод строки
nl_opt	Ноль или более переводов строки
and_or	Конвейеры, связанные через && / ||
pipeline	Команды, связанные через |
command	Простая команда, присваивание, [...], составная команда, функция или local
simple_command	Имя, аргументы и редиректы в конце
assignment	Присваивание NAME= / NAME=value
test_command	Проверка [...]
if_clause, while_clause, until_clause, for_clause, select_clause, case_clause	Составные команды
function_def, brace_group, local_stmt	Функция, группа команд и local
cmdsubst	Подстановка $(list)
sep	Разделитель перед then / do: ; или перевод строки

Таблица 2 – Основные синтаксические элементы

Стартовый символ — program. Иерархия правил сверху вниз: program → list → stmt_seq → and_or → pipeline → command. В правилах, которые описывают допустимый язык, семантических действий нет: анализатор не вычисляет значения команд.

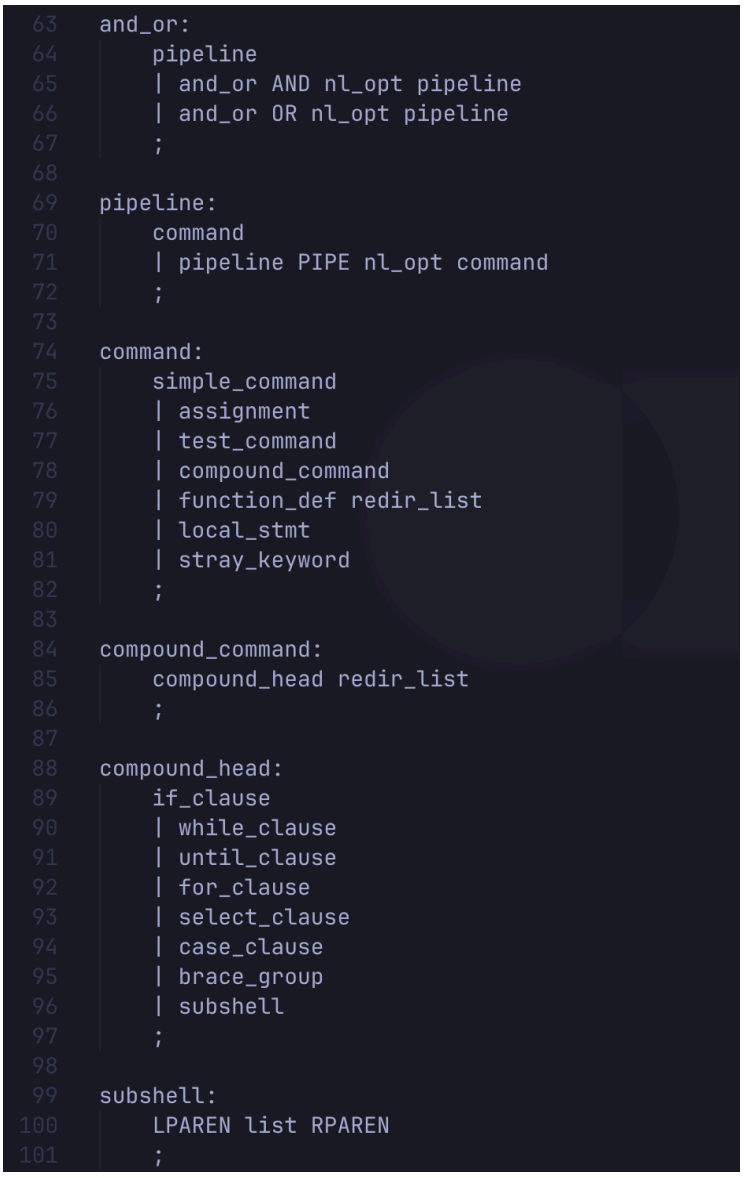


```
38  program:
39      SHEBANG list
40      ;
41
42  list:
43      nl_opt
44      | nl_opt stmt_seq
45      | nl_opt stmt_seq separator
46      ;
47
48  stmt_seq:
49      and_or
50      | stmt_seq separator and_or
51      ;
52
53  separator:
54      SEMI nl_opt
55      | AMP nl_opt
56      | NL nl_opt
57      ;
58
59  nl_opt:
60      | nl_opt NL
61      ;
```

Рисунок 1 – Начало грамматики

Программа обязана начинаться с шебанга. За ним идёт список команд. Файл из одного шебанга принадлежит языку, пустой файл — нет.

separator завершает команду: ; и перевод строки равносильны, & помечает фон. nl_opt разрешает пустые строки и перенос &&, ||, | без пустой команды. Дальше грамматика сужается до одной команды.



```
and_or:
    pipeline
    | and_or AND nl_opt pipeline
    | and_or OR nl_opt pipeline
    ;

pipeline:
    command
    | pipeline PIPE nl_opt command
    ;

command:
    simple_command
    | assignment
    | test_command
    | compound_command
    | function_def redir_list
    | local_stmt
    | stray_keyword
    ;

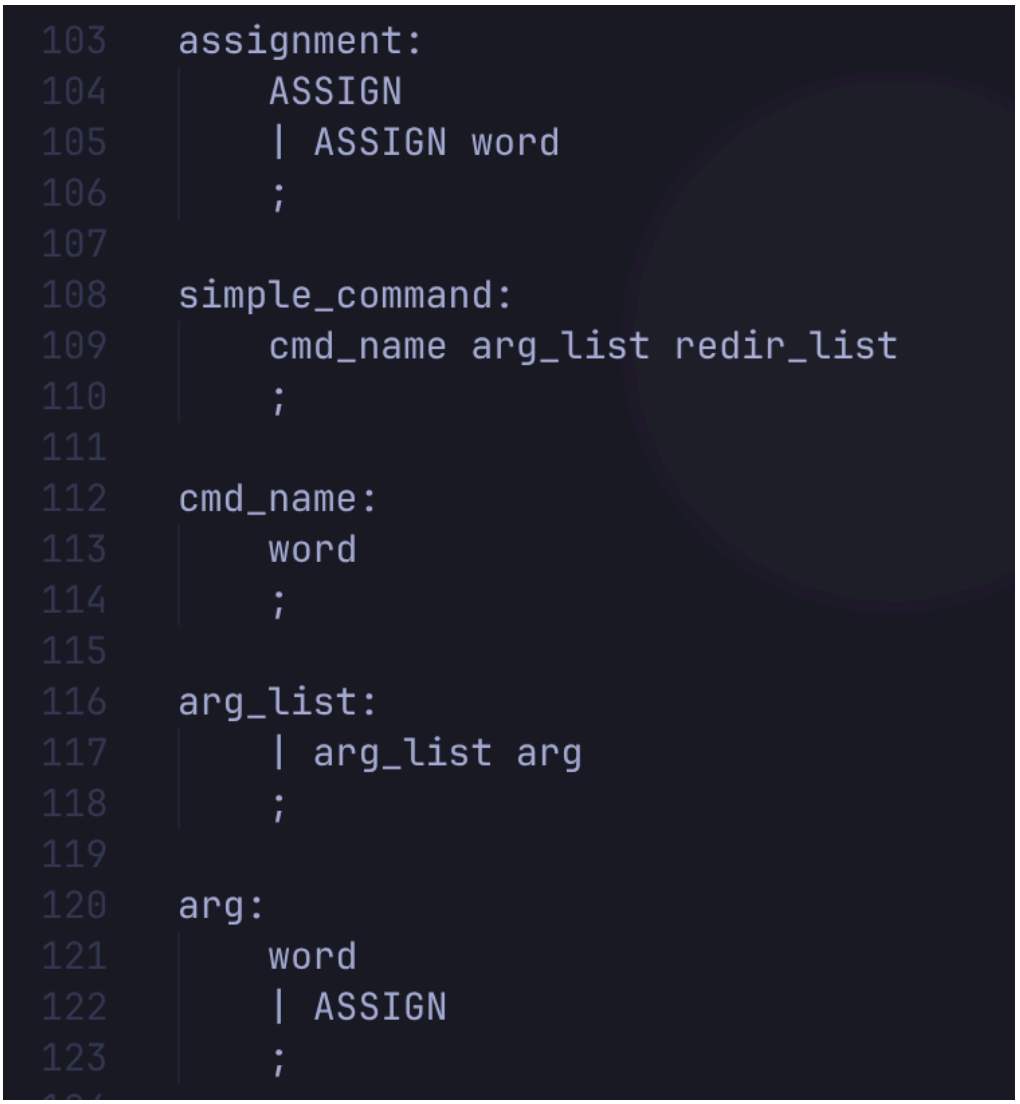
compound_command:
    compound_head redir_list
    ;

compound_head:
    if_clause
    | while_clause
    | until_clause
    | for_clause
    | select_clause
    | case_clause
    | brace_group
    | subshell
    ;

subshell:
    LPAREN list RPAREN
    ;
```

Рисунок 2 – Списки && и ||, конвейер и составная команда

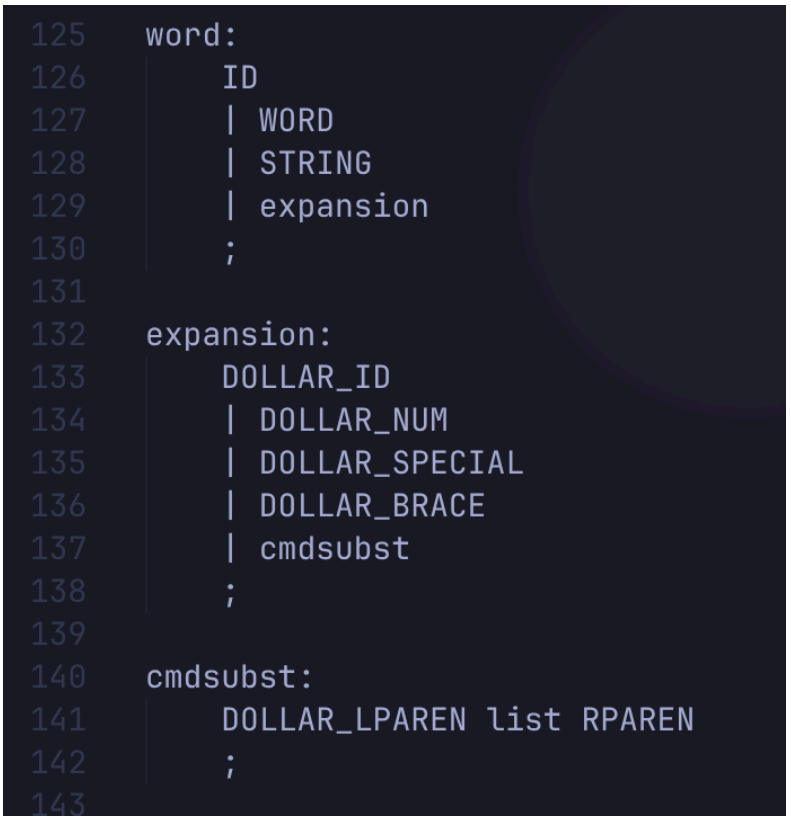
and_or — это цепочка конвейеров через && и ||. pipeline — цепочка команд через |. После &&, || и | может быть пустая строка: её забирает nl_opt. Конвейер стоит ниже списка, поэтому в записи a && b | c сначала связывается b | c, и только потом результат соединяется с a через &&. Одно звено такой цепочки — command: простая команда, присваивание, проверка [...], составная команда, функция или local. Составная команда — это if, while, until, for, select, case, группа или подшелл, и после неё может стоять перенаправление. Правило stray_keyword ловит then и do не на своём месте.



```
103  assignment:
104      ASSIGN
105      | ASSIGN word
106      ;
107
108  simple_command:
109      cmd_name arg_list redir_list
110      ;
111
112  cmd_name:
113      word
114      ;
115
116  arg_list:
117      | arg_list arg
118      ;
119
120  arg:
121      word
122      | ASSIGN
123      ;
```

Рисунок 3 – Присваивание и простая команда

Присваивание — это NAME= или NAME= и следующее слово, например FOO= и FOO=1. Простая команда состоит из имени, аргументов и перенаправлений в конце. Имя команды — то же слово, что и аргумент: echo, строка в кавычках или подстановка. Аргументом может быть и присваивание, поэтому echo FOO=1 остаётся обычной командой. Запись FOO=1 echo x сюда не входит.



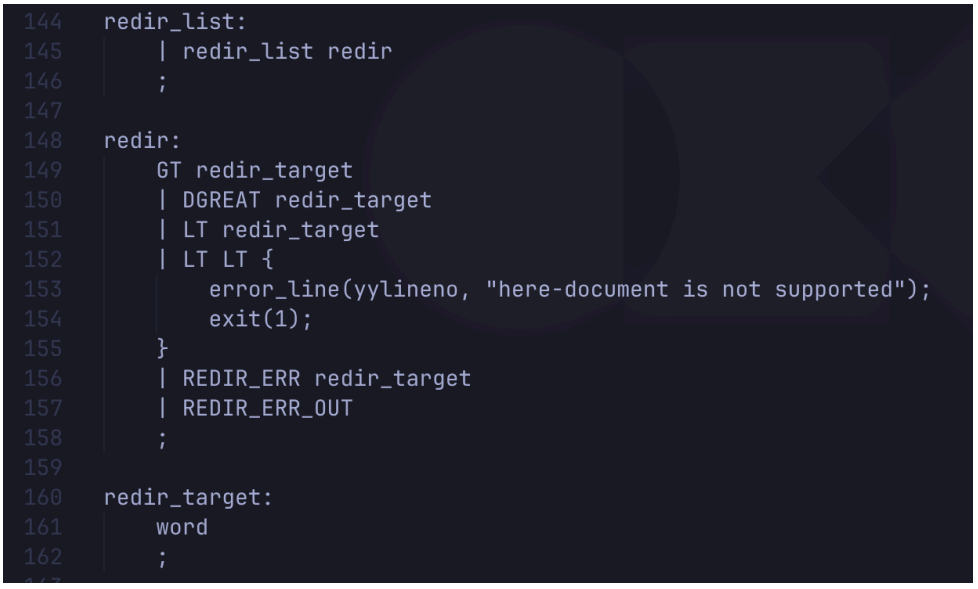
```
word:
    ID
    | WORD
    | STRING
    | expansion
    ;

expansion:
    DOLLAR_ID
    | DOLLAR_NUM
    | DOLLAR_SPECIAL
    | DOLLAR_BRACE
    | cmdsubst
    ;

cmdsubst:
    DOLLAR_LPAREN list RPAREN
    ;
```

Рисунок 4 – Слово и подстановки

Слово — это имя, обычный набор символов, строка в кавычках или подстановка. Подстановка бывает такой: $имя, $1, $#, $@, $*, $$, $?, ${имя} и $(...). Внутри $(...) разбирается тот же список команд, что и в теле файла, только без шебанга.



```
redir_list:
    | redir_list redir
    ;

redir:
    GT redir_target
    | DGREAT redir_target
    | LT redir_target
    | LT LT {
        error_line(yylineno, "here-document is not supported");
        exit(1);
    }
    | REDIR_ERR redir_target
    | REDIR_ERR_OUT
    ;

redir_target:
    word
    ;
```

Рисунок 5 – Перенаправления

Перенаправлений может не быть. Допустимы >, >>, <, 2> и 2>&1. Они стоят в конце простой команды, проверки [...], составной команды и объявления функции. Цель >, >>, < и 2> — слово: имя, обычное слово, строка или подстановка. Два знака < подряд — это here-документ. Такое правило отвергает его и печатает here-document is not supported.

```
164  test_command:
165      LBRACK arg_list RBRACK redir_list
166      | LBRACK LBRACK {
167          error_line(yylineno, "[[ is not supported");
168          exit(1);
169      }
170      ;
```

Рисунок 6 – Проверка в квадратных скобках

Проверка [...] — это слова в квадратных скобках, и у неё самой может быть перенаправление. Пример: [-f "$1"] > /tmp/f. Между скобками может не быть слов, как в [ ], или несколько слов, как в [-f file]. Знаки -f, = и -lt остаются обычными словами. Две скобки подряд, [[, не принимаются: печатается пояснение [[is not supported.

```
172  if_clause:
173      IF and_or sep THEN list FI
174      | IF and_or sep THEN list elif_parts FI
175      | IF and_or sep THEN list ELSE list FI
176      | IF and_or sep THEN list elif_parts ELSE list FI
177      | IF and_or THEN {
178          error_line(yylineno, "missing ';' or newline before then");
179          exit(1);
180      }
181      ;
182
183  elif_parts:
184      ELIF and_or sep THEN list
185      | elif_parts ELIF and_or sep THEN list
186      | ELIF and_or THEN {
187          error_line(yylineno, "missing ';' or newline before then");
188          exit(1);
189      }
190      ;
```

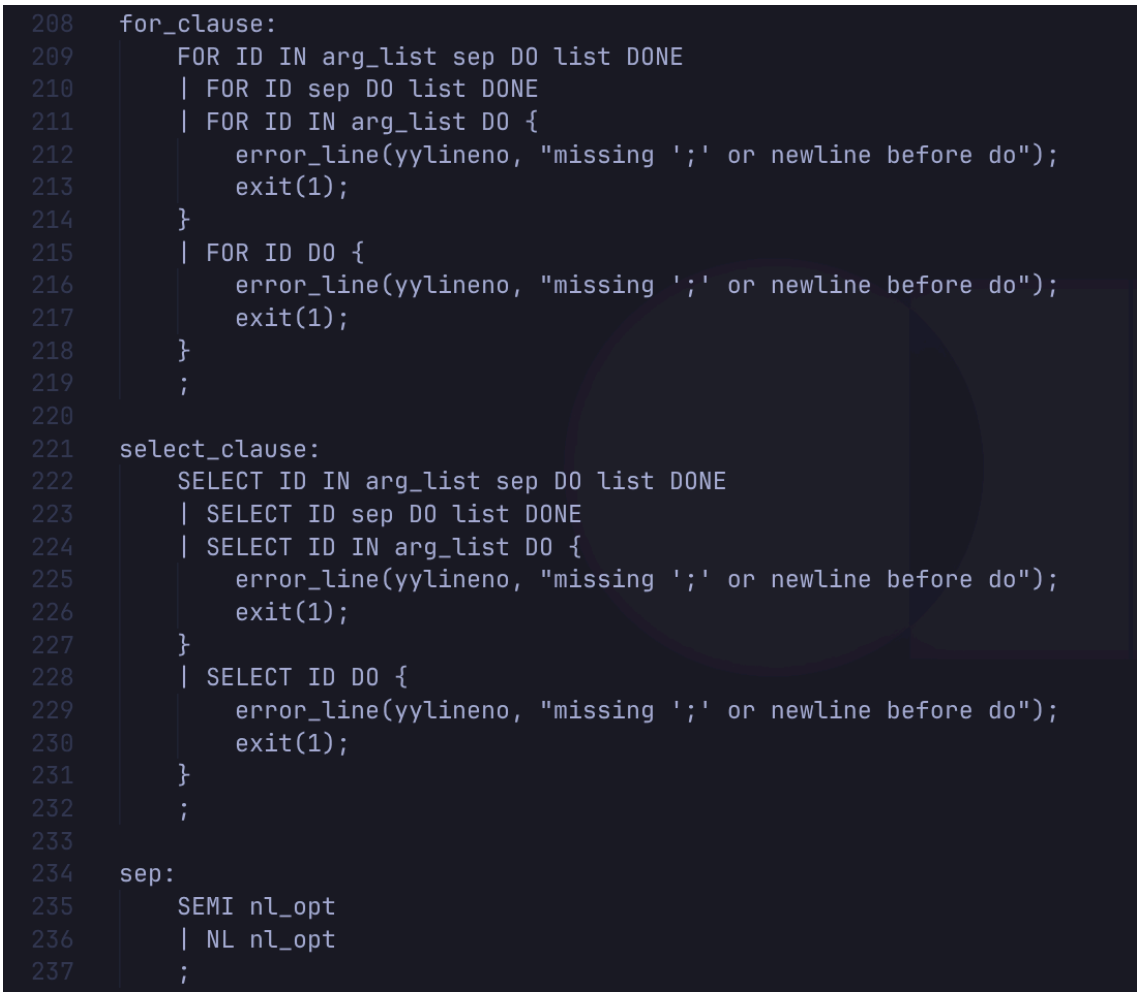
Рисунок 7 – Условие if и elif

Условие if и elif — одна команда, конвейер или цепочка && и ||, а не только [...]. Запись if true; then языку принадлежит. Две команды через ; одним условием не считаются. Перед then нужны ; или перевод строки. Без них, как в if true then, печатается пояснение. После then идёт список команд. Веток elif может быть несколько, и перед then в каждой из них снова нужны ; или перевод строки. Ветка else необязательна. Конструкцию закрывает fi.

```
192  while_clause:
193      WHILE and_or sep DO list DONE
194      | WHILE and_or DO {
195          error_line(yylineno, "missing ';' or newline before do");
196          exit(1);
197      }
198      ;
199
200  until_clause:
201      UNTIL and_or sep DO list DONE
202      | UNTIL and_or DO {
203          error_line(yylineno, "missing ';' or newline before do");
204          exit(1);
205      }
206      ;
```

Рисунок 8 – Циклы while и until

Условие while и until — одна команда, конвейер или цепочка && и ||. Тело стоит между do и done. Перед do нужны ; или перевод строки. Без них, как в while true do, печатается пояснение.



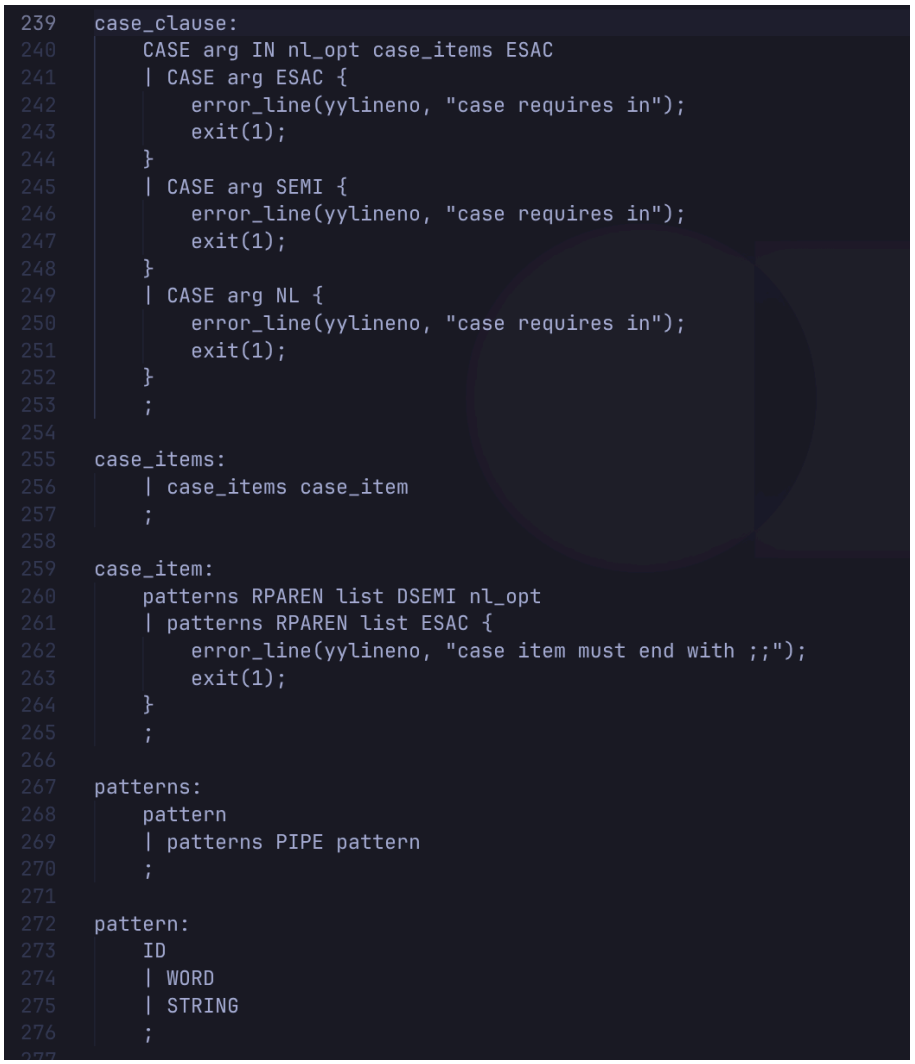
```
for_clause:
    FOR ID IN arg_list sep DO list DONE
    | FOR ID sep DO list DONE
    | FOR ID IN arg_list DO {
        error_line(yylineno, "missing ';' or newline before do");
        exit(1);
    }
    | FOR ID DO {
        error_line(yylineno, "missing ';' or newline before do");
        exit(1);
    }
    ;

select_clause:
    SELECT ID IN arg_list sep DO list DONE
    | SELECT ID sep DO list DONE
    | SELECT ID IN arg_list DO {
        error_line(yylineno, "missing ';' or newline before do");
        exit(1);
    }
    | SELECT ID DO {
        error_line(yylineno, "missing ';' or newline before do");
        exit(1);
    }
    ;

sep:
    SEMI nl_opt
    | NL nl_opt
    ;
```

Рисунок 9 – Циклы for и select

У for и select есть форма со списком in и форма без in. Со списком пишут for name in a b; do ... done, без списка — for name; do ... done. У select те же две формы. Тело стоит между do и done. Перед do нужны ; или перевод строки. Это правило sep: ; или перевод строки, и после них могут идти пустые строки. Тем же разделителем условие отделяется от then. Без разделителя, как в for name do, печатается пояснение.



```
case_clause:
    CASE arg IN nl_opt case_items ESAC
    | CASE arg ESAC {
        error_line(yylineno, "case requires in");
        exit(1);
    }
    | CASE arg SEMI {
        error_line(yylineno, "case requires in");
        exit(1);
    }
    | CASE arg NL {
        error_line(yylineno, "case requires in");
        exit(1);
    }
    ;

case_items:
    | case_items case_item
    ;

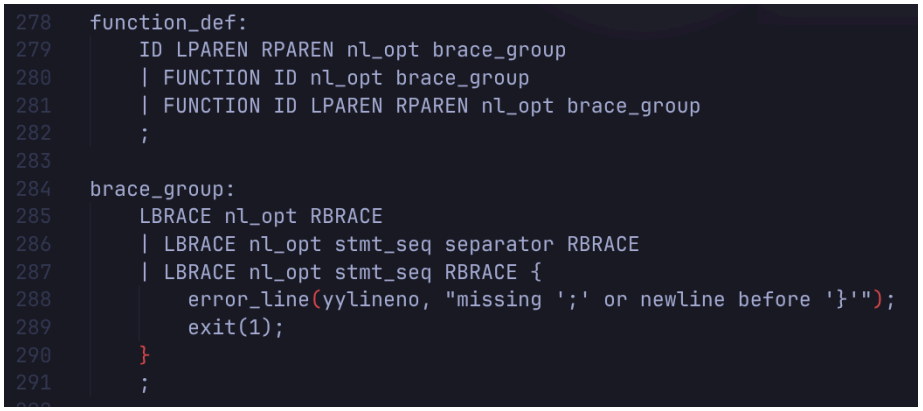
case_item:
    patterns RPAREN list DSEMI nl_opt
    | patterns RPAREN list ESAC {
        error_line(yylineno, "case item must end with ;;");
        exit(1);
    }
    ;

patterns:
    pattern
    | patterns PIPE pattern
    ;

pattern:
    ID
    | WORD
    | STRING
    ;
```

Рисунок 10 – Образцы case

Образец в case — это имя, слово или строка. Несколько образцов в одной ветке пишутся через |, как в a|b). Если нет in или ветка доходит до esac без ;;, печатается пояснение.



```
function_def:
    ID LPAREN RPAREN nl_opt brace_group
    | FUNCTION ID nl_opt brace_group
    | FUNCTION ID LPAREN RPAREN nl_opt brace_group
    ;

brace_group:
    LBRACE nl_opt RBRACE
    | LBRACE nl_opt stmt_seq separator RBRACE
    | LBRACE nl_opt stmt_seq RBRACE {
        error_line(yylineno, "missing ';' or newline before '}'");
        exit(1);
    }
    ;
```

Рисунок 11 – Функция и её тело

Функция задаётся тремя способами: name() { ...; }, function name { ...; } и function name() { ...; }. Тело может быть пустым. Если внутри есть команды, перед } нужны ; или перевод строки, иначе печатается пояснение. Та же группа в фигурных скобках может быть отдельной командой. После объявления функции может стоять перенаправление.

```
304  local_stmt:
305      LOCAL local_names
306      ;
307
308  local_names:
309      local_item
310      | local_names local_item
311      ;
312
313  local_item:
314      ID
315      | ASSIGN
316      | ASSIGN word
317      ;
```

Рисунок 12 – Объявление local

У local должен быть хотя бы один аргумент. Это имя (a), присваивание без значения (a=) или со значением (a=1). Несколько аргументов пишутся подряд: local a b=1. Внутри функции стоит объявление или нет, грамматика не проверяет.

2.4 Диагностические правила

Обычная синтаксическая ошибка даёт только номер строки (Error: line N), потому что bison не знает, что именно пошло не так. Чтобы для типичных ошибок выдавать пояснение, в грамматику добавлены диагностические правила (далее «псевдоправила»). Такое правило описывает не допустимую конструкцию, а типичную ошибочную запись.

Например, правило IF and_or THEN срабатывает, когда после условия if сразу идёт then, без ; или перевода строки. Остальные псевдоправила приведены в таблице 3.

Правило	Ошибочная запись	Сообщение
redir: LT LT	cat <<EOF	here-document is not supported
test_command: LBRACK LBRACK	[[-f x]]	[[is not supported
if_clause: IF and_or THEN, elif_parts: ELIF and_or THEN	if true then	missing ‘;’ or newline before then
while_clause, until_clause, for_clause, select_clause	while true do, for i in a b do	missing ‘;’ or newline before do
case_clause: CASE arg ESAC / SEMI / NL	case $x esac	case requires in
case_item: patterns RPAREN list ESAC	a) echo ok esac	case item must end with ;;
brace_group: LBRACE nl_opt stmt_seq RBRACE	f() { echo a }	missing ‘;’ or newline before ‘}’
stray_keyword: THEN, DO	then или do не на месте	unexpected then, unexpected do

Таблица 2 – Основные синтаксические элементы

2.5 Возникшие конфликты и их исправление

Команда bison -d -y -v pars_bash.y сообщает один конфликт сдвига и свёртки. Это значит, что в некотором состоянии автомата при одной и той же следующей лексеме возможны два действия: сдвинуть лексему и продолжить начатое правило или выполнить свёртку и считать это правило законченным. Конфликтов двух свёрток нет.

Конфликт возникает в правиле local на лексеме ID, которая идёт сразу после ASSIGN. Аргументом local может быть имя, присваивание без значения (a=) или присваивание со значением (a=1), см. рисунок 12. Спорный текст показан на рисунке 13: между a= и b стоит пробел.

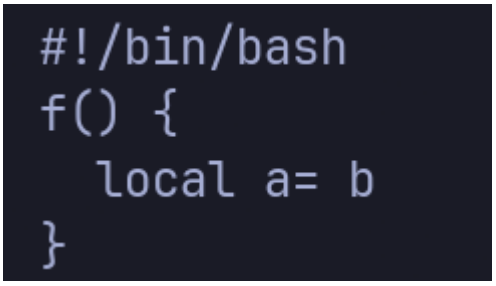


Рисунок 13 – Код, на котором виден конфликт в local

Сканер пробелы отбрасывает, поэтому парсер видит ASSIGN (a=), а следом ID (b). Из-за этого он не может определить, относится ли b к a=. Возможны два хода:

- Сдвиг. Лексема b читается как значение по правилу local_item → ASSIGN word. Получается один аргумент a= b.
- Свёртка. Пустое a= закрывается по правилу local_item → ASSIGN, а b становится следующим именем по правилу local_item → ID. Получается два аргумента.

Оба хода приводят к успешному разбору, поэтому текст принимается при любом выборе. Различается только дерево разбора: для этого текста грамматика неоднозначна. Распознаватель решает лишь вопрос принадлежности текста языку и значений не вычисляет, поэтому ответ

программы от выбора не зависит. В таком споре bison по умолчанию выбирает сдвиг, и local a= b разбирается как один аргумент. В настоящем Bash это два аргумента, но для распознавателя разница не видна, поэтому грамматика не менялась. Без пробела, в local a=b, сканер отдаёт одну лексему ASSIGN, и спора нет.

На рисунке 14 сверху показана сборка с предупреждением о конфликте, снизу тот же конфликт с флагом -Wcounterexamples. Bison приводит пример ASSIGN • ID и две ветви разбора: сдвиг идёт по правилу ASSIGN word, свёртка — по правилу ASSIGN.

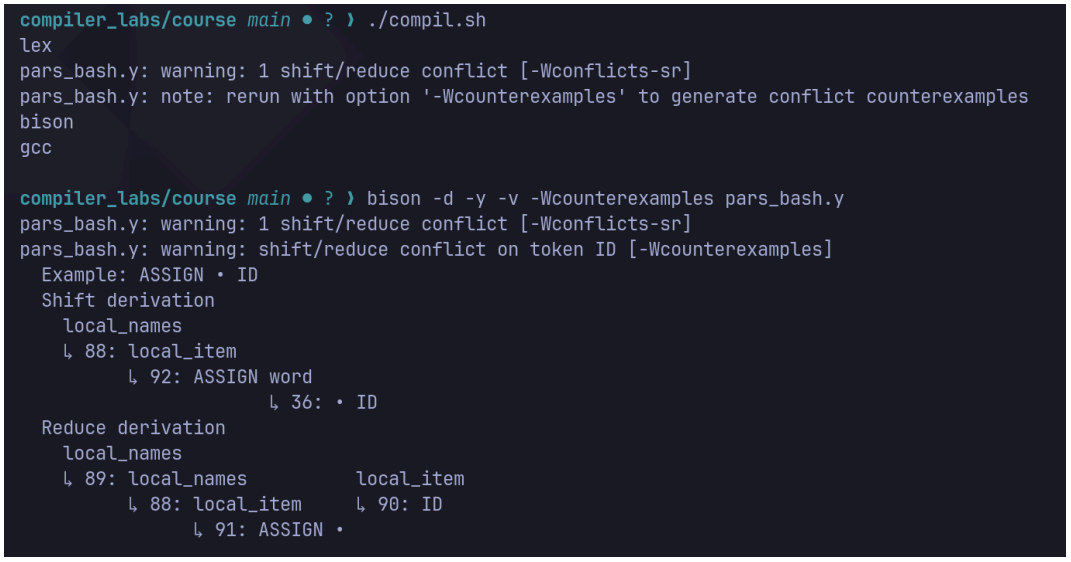


Рисунок 14 – Конфликт сдвига и свёртки в local на токене ID

2.6 Работоспособность парсера

Если текст принадлежит подмножеству, программа ничего не печатает и завершается с кодом 0. Если найдена ошибка, печатается Error: line N и код возврата равен 1. N — номер строки, считая с 1. После первой ошибки разбор останавливается.

Для нескольких типичных случаев к номеру добавляется короткое пояснение: Error: line N: text. Такие пояснения есть у шебанга, here-документа, [[, пропущенного разделителя перед then, do и }, неожиданных then и do, case без in и ветки case без ;;. Остальные ошибки сообщаются только номером строки. На рисунке 19 ветка if закрыта словом done, поэтому в сообщении один номер строки.

На рисунках 15 и 16 два принимаемых сценария. Сообщений нет, код возврата равен 0.

```
compiler_labs/course main ● ? ❱ cat tests/accept/12-if-elif-else.sh
#!/bin/bash
if [ "$1" = a ]; then
  echo a
elif [ "$1" = b ]; then
  echo b
else
  echo other
fi

compiler_labs/course main ● ? ❱ ./bash_parser tests/accept/12-if-elif-else.sh

compiler_labs/course main ● ? ❱
```

Рисунок 15 – Принимаемый сценарий: if, elif и else

```
compiler_labs/course main ● ? ❱ cat tests/accept/19-local.sh
#!/bin/bash
f() {
  local name
  local name="$1"
  local a b=1
}

compiler_labs/course main ● ? ❱ ./bash_parser tests/accept/19-local.sh

compiler_labs/course main ● ? ❱
```

Рисунок 16 – Принимаемый сценарий: функция и local

На рисунке 17 файл без шебанга: Error: line 1: missing shebang, код возврата равен 1. На рисунке 18 here-документ: Error: line 2: here-document is not supported, код возврата равен 1.

```
compiler_labs/course main ● ? ❱ cat tests/reject/no-shebang.sh
echo hello

compiler_labs/course main ● ? ❱ ./bash_parser tests/reject/no-shebang.sh
Error: line 1: missing shebang

compiler_labs/course main ● ? ×
```

Рисунок 17 – Отказ: нет шебанга

```
compiler_labs/course main ● ? × cat tests/reject/here-doc.sh
#!/bin/bash
cat <<EOF
x
EOF

compiler_labs/course main ● ? ❱ ./bash_parser tests/reject/here-doc.sh
Error: line 2: here-document is not supported

compiler_labs/course main ● ? ×
```

Рисунок 18 – Отказ: here-документ не поддерживается

На рисунке 19 обычная синтаксическая ошибка: Error: line 4, код возврата равен 1.

```
compiler_labs/course main ● ? ❱ cat tests/reject/if-wrong-close.sh
#!/bin/bash
if [ a ]; then
  echo x
done

compiler_labs/course main ● ? ❱ ./bash_parser tests/reject/if-wrong-close.sh
Error: line 4
```

Рисунок 19 – Отказ: ошибка на строке 4

```
compiler_labs/course main ● ? ❱ cat ./tests/accept/30-overview.sh
#!/bin/bash
name="World"
count=0
if [ $# -lt 1 ]; then echo usage; fi
echo $name | wc -l > /tmp/out
greet() {
  local name="$1"
  echo ${name}
}
for item in a b; do greet $item; done
case $item in
a|b) echo ok ;;
*) echo other ;;
esac

compiler_labs/course main ● ? ❱ ./bash_parser ./tests/accept/30-overview.sh

compiler_labs/course main ● ? ❱
```

Рисунок 20 – Принимаемый сценарий: большой файл

На рисунке 20 показан большой принимаемый сценарий: присваивания, if, конвейер с перенаправлением, функция с local, for и case. Сообщений нет, код возврата равен 0.

3 РЕЗУЛЬТАТЫ

В ходе курсовой работы сделано следующее:

- Описано учебное подмножество Bash и отдельно указано, какие конструкции полного языка в него не входят.

- На flex реализован лексический анализатор: шебанг, ключевые слова, слова, присваивания, операторы, кавычки и подстановки.

- На bison в режиме yacc реализован синтаксический анализатор. При сборке остаётся один конфликт в правиле local. На ответ программы он не влияет.

- Собран распознаватель bash_parser. Верный текст он принимает без сообщений и завершается с кодом 0. При первой ошибке печатает Error: line N и завершается с кодом 1. Для шебанга, here-документа и условия без [...] к номеру строки добавляется короткое пояснение.

Для типичных ошибок (шебанг, here-документ, [[, пропущенный разделитель перед then, do и }, лишние then и do, case без in, ветка case без ;;) к номеру строки добавляется короткое пояснение.

4 ВЫВОД

Цель работы достигнута. Для учебного подмножества Bash составлена собственная контекстно-свободная грамматика и на flex и yacc собран распознаватель. Программа отвечает, принадлежит ли текст подмножеству, и сообщает строку первой ошибки.

Полный язык в одну такую грамматику не укладывается. Here-документ, арифметика и сложные подстановки читаются по своим правилам. Конфликт в local оставлен: на приём и отказ текста он не влияет.

ПРИЛОЖЕНИЕ 1

```
%{
#include <stdio.h>
#include <stdlib.h>
#include "y.tab.h"

extern void error_line(int line, const char *msg);

static int read_single_quote(void);
static int read_double_quote(void);
%}

%option yylineno noyywrap nounput
%x BODY

ID      [A-Za-z_][A-Za-z0-9_]*
WORD    [^ \t\n|&;<>(){}[\]"'`$\\]+

%%

"#!/bin/bash"[ \t]*\n  { BEGIN(BODY); return SHEBANG; }
"#!/bin/bash"[ \t]*    { BEGIN(BODY); return SHEBANG; }
"#!/bin/bash".*        { error_line(1, "extra text in shebang"); exit(1); }
.                      { error_line(1, "missing shebang"); exit(1); }
\n                     { error_line(1, "missing shebang"); exit(1); }
<INITIAL><<EOF>>       { error_line(1, "missing shebang"); exit(1); }

<BODY>[ \t]+                    { }
<BODY>#.*                       { }

<BODY>"if"                      { return IF; }
<BODY>"then"                    { return THEN; }
<BODY>"elif"                    { return ELIF; }
<BODY>"else"                    { return ELSE; }
<BODY>"fi"                      { return FI; }
<BODY>"while"                   { return WHILE; }
<BODY>"until"                   { return UNTIL; }
<BODY>"do"                      { return DO; }
<BODY>"done"                    { return DONE; }
<BODY>"for"                     { return FOR; }
<BODY>"select"                  { return SELECT; }
<BODY>"in"                      { return IN; }
<BODY>"case"                    { return CASE; }
<BODY>"esac"                    { return ESAC; }
<BODY>"function"                { return FUNCTION; }
<BODY>"local"                   { return LOCAL; }

<BODY>"2>&1"                    { return REDIR_ERR_OUT; }
<BODY>"2>"                      { return REDIR_ERR; }
<BODY>">>"                      { return DGREAT; }
<BODY>"&&"                      { return AND; }
<BODY>"||"                      { return OR; }
<BODY>";;"                      { return DSEMI; }

<BODY>"|"                       { return PIPE; }
```

```
<BODY>"&"                       { return AMP; }
<BODY>";"                       { return SEMI; }
<BODY>"<"                       { return LT; }
<BODY>">"                       { return GT; }
<BODY>"("                       { return LPAREN; }
<BODY>")"                       { return RPAREN; }
<BODY>"{"                       { return LBRACE; }
<BODY>"}"                       { return RBRACE; }
<BODY>"["                       { return LBRACK; }
<BODY>"]"                       { return RBRACK; }

<BODY>"$("                      { return DOLLAR_LPAREN; }
<BODY>"${"{ID}"}"               { return DOLLAR_BRACE; }
<BODY>"${"                      { error_line(yylineno, NULL); exit(1); }
<BODY>"#"|"@"|"* "|""|"?"  { return DOLLAR_SPECIAL; }
<BODY>"$"{ID}                   { return DOLLAR_ID; }
<BODY>"$"[0-9]+                 { return DOLLAR_NUM; }
<BODY>"$"                       { error_line(yylineno, NULL); exit(1); }

<BODY>{ID}={WORD}               { return ASSIGN; }
<BODY>{ID}=                     { return ASSIGN; }
<BODY>{ID}                      { return ID; }
<BODY>{WORD}                    { return WORD; }

<BODY>'                         { return read_single_quote(); }
<BODY>\"                        { return read_double_quote(); }

<BODY>\n                        { return NL; }

<BODY>.                         { error_line(yylineno, NULL); exit(1); }

%%

static int read_single_quote(void)
{
    int line = yylineno;
    int c;

    while ((c = input()) != 0) {
        if (c == '\'')
            return STRING;
        if (c == '\n') {
            error_line(line, NULL);
            exit(1);
        }
    }
    error_line(line, NULL);
    exit(1);
}

static int read_double_quote(void)
{
    int line = yylineno;
    int c;
    int escaped = 0;
```

```
    while ((c = input()) != 0) {
        if (c == '\n') {
            if (!escaped) {
                error_line(line, NULL);
                exit(1);
            }
            line++;
            escaped = 0;
            continue;
        }
        if (escaped) {
            escaped = 0;
            continue;
        }
        if (c == '\\') {
            escaped = 1;
            continue;
        }
        if (c == '"')
            return STRING;
    }
    error_line(line, NULL);
    exit(1);
}
```

ПРИЛОЖЕНИЕ 2

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
extern int yylineno;
extern FILE *yyin;

void error_line(int line, const char *msg)
{
    if (msg)
        fprintf(stderr, "Error: line %d: %s\n", line, msg);
    else
        fprintf(stderr, "Error: line %d\n", line);
}

void yyerror(char *msg)
{
    (void)msg;
    error_line(yylineno, NULL);
}
%}

%token SHEBANG
%token IF THEN ELIF ELSE FI
%token WHILE UNTIL DO DONE
%token FOR SELECT IN CASE ESAC FUNCTION LOCAL
%token ID WORD STRING ASSIGN
%token DOLLAR_ID DOLLAR_NUM DOLLAR_SPECIAL DOLLAR_BRACE DOLLAR_LPAREN
%token LBRACK RBRACK LBRACE RBRACE LPAREN RPAREN
%token PIPE AND OR AMP SEMI DSEMI NL
%token LT GT DGREAT REDIR_ERR REDIR_ERR_OUT

%start program

%%
/* шебанг и команды */
program:
    SHEBANG list
    ;

/* тело: файл, then, do; можно пустое */
list:
    nl_opt
    | nl_opt stmt_seq
    | nl_opt stmt_seq separator
    ;

/* команды через separator */
stmt_seq:
    and_or
    | stmt_seq separator and_or
    ;
```

```
/* ;, & или перевод строки */
separator:
    SEMI nl_opt
    | AMP nl_opt
    | NL nl_opt
    ;

/* лишние переводы строки */
nl_opt:
    | nl_opt NL
    ;

/* && и ||; ещё условие if, while, until */
and_or:
    pipeline
    | and_or AND nl_opt pipeline
    | and_or OR nl_opt pipeline
    ;

/* конвейер | */
pipeline:
    command
    | pipeline PIPE nl_opt command
    ;

/* одна команда */
command:
    simple_command
    | assignment
    | test_command
    | compound_command
    | function_def redir_list
    | local_stmt
    | stray_keyword
    ;

/* составная команда и редиректы */
compound_command:
    compound_head redir_list
    ;

/* if, циклы, case, группа, подшелл */
compound_head:
    if_clause
    | while_clause
    | until_clause
    | for_clause
    | select_clause
    | case_clause
    | brace_group
    | subshell
    ;

/* ( команды ) */
subshell:
```

```
    LPAREN list RPAREN
    ;

/* NAME= или NAME=слово */
assignment:
    ASSIGN
    | ASSIGN word
    ;

/* имя, аргументы, редиректы */
simple_command:
    cmd_name arg_list redir_list
    ;

/* имя команды */
cmd_name:
    word
    ;

/* аргументы, можно пусто */
arg_list:
    | arg_list arg
    ;

/* слово или NAME=value */
arg:
    word
    | ASSIGN
    ;

/* имя, слово, строка, подстановка */
word:
    ID
    | WORD
    | STRING
    | expansion
    ;

/* $имя, $1, ${имя}, $# $@ $* $$ $? */
expansion:
    DOLLAR_ID
    | DOLLAR_NUM
    | DOLLAR_SPECIAL
    | DOLLAR_BRACE
    | cmdsubst
    ;

/* $(...) */
cmdsubst:
    DOLLAR_LPAREN list RPAREN
    ;

/* редиректы */
redir_list:
    | redir_list redir
    ;
```

```
/* > >> < 2> 2>&1; << нельзя */
redir:
    GT redir_target
    | DGREAT redir_target
    | LT redir_target
    | LT LT {
        error_line(yylineno, "here-document is not supported");
        exit(1);
    }
    | REDIR_ERR redir_target
    | REDIR_ERR_OUT
    ;

redir_target:
    word
    ;

/* [ слова ]; [[ нельзя */
test_command:
    LBRACK arg_list RBRACK redir_list
    | LBRACK LBRACK {
        error_line(yylineno, "[[ is not supported");
        exit(1);
    }
    ;

/* if; перед then — ; или перевод строки */
if_clause:
    IF and_or sep THEN list FI
    | IF and_or sep THEN list elif_parts FI
    | IF and_or sep THEN list ELSE list FI
    | IF and_or sep THEN list elif_parts ELSE list FI
    | IF and_or THEN {
        error_line(yylineno, "missing ';' or newline before then");
        exit(1);
    }
    ;

/* ветки elif */
elif_parts:
    ELIF and_or sep THEN list
    | elif_parts ELIF and_or sep THEN list
    | ELIF and_or THEN {
        error_line(yylineno, "missing ';' or newline before then");
        exit(1);
    }
    ;

/* while do ... done */
while_clause:
    WHILE and_or sep DO list DONE
    | WHILE and_or DO {
        error_line(yylineno, "missing ';' or newline before do");
        exit(1);
    }
```

```
    ;

/* until, как while */
until_clause:
    UNTIL and_or sep DO list DONE
    | UNTIL and_or DO {
        error_line(yylineno, "missing ';' or newline before do");
        exit(1);
    }
    ;

/* for со списком in и без */
for_clause:
    FOR ID IN arg_list sep DO list DONE
    | FOR ID sep DO list DONE
    | FOR ID IN arg_list DO {
        error_line(yylineno, "missing ';' or newline before do");
        exit(1);
    }
    | FOR ID DO {
        error_line(yylineno, "missing ';' or newline before do");
        exit(1);
    }
    ;

/* select, как for */
select_clause:
    SELECT ID IN arg_list sep DO list DONE
    | SELECT ID sep DO list DONE
    | SELECT ID IN arg_list DO {
        error_line(yylineno, "missing ';' or newline before do");
        exit(1);
    }
    | SELECT ID DO {
        error_line(yylineno, "missing ';' or newline before do");
        exit(1);
    }
    ;

sep:
    SEMI nl_opt
    | NL nl_opt
    ;

/* case; нужно in */
case_clause:
    CASE arg IN nl_opt case_items ESAC
    | CASE arg ESAC {
        error_line(yylineno, "case requires in");
        exit(1);
    }
    | CASE arg SEMI {
        error_line(yylineno, "case requires in");
        exit(1);
    }
    | CASE arg NL {
```

```
        error_line(yylineno, "case requires in");
        exit(1);
    }
    ;

case_items:
    | case_items case_item
    ;

/* образец ) тело ;; */
case_item:
    patterns RPAREN list DSEMI nl_opt
    | patterns RPAREN list ESAC {
        error_line(yylineno, "case item must end with ;;");
        exit(1);
    }
    ;

/* несколько образцов через | */
patterns:
    pattern
    | patterns PIPE pattern
    ;

/* имя, слово или строка */
pattern:
    ID
    | WORD
    | STRING
    ;

/* name(), function name, function name() */
function_def:
    ID LPAREN RPAREN nl_opt brace_group
    | FUNCTION ID nl_opt brace_group
    | FUNCTION ID LPAREN RPAREN nl_opt brace_group
    ;

/* { }; перед } нужен ; или перевод строки */
brace_group:
    LBRACE nl_opt RBRACE
    | LBRACE nl_opt stmt_seq separator RBRACE
    | LBRACE nl_opt stmt_seq RBRACE {
        error_line(yylineno, "missing ';' or newline before '}'");
        exit(1);
    }
    ;

/* then или do не на месте */
stray_keyword:
    THEN {
        error_line(yylineno, "unexpected then");
        exit(1);
    }
    | DO {
        error_line(yylineno, "unexpected do");
```

```
        exit(1);
    }
    ;

/* local */
local_stmt:
    LOCAL local_names
    ;

local_names:
    local_item
    | local_names local_item
    ;

local_item:
    ID
    | ASSIGN
    | ASSIGN word
    ;

%%

int main(int argc, char **argv)
{
    int result;

    if (argc > 1) {
        yyin = fopen(argv[1], "r");
        if (!yyin) {
            perror(argv[1]);
            return 1;
        }
    }
    result = yyparse();
    if (argc > 1)
        fclose(yyin);
    return result;
}
```